I)
Investigación Arquitectura de Red

Concepto	Definición simple	Nivel técnico breve	Ejemplo real
Modelo OSI	Un mapa de 7 capas que explica cómo viajan los datos en una red, de hardware a apps.	Framework conceptual que divide la comunicación en capas: Física, Enlace, Red, Transporte, Sesión, Presentación, Aplicación. Cada capa solo habla con la de arriba/abajo.	Cuando abres una web: la capa Aplicación (HTTP) pide datos → Transporte (TCP) los divide en paquetes → Red (IP) los enruta → Física los envía como señal.
Subred (subnet)	Dividir una red grande en redes más pequeñas para organizarlas mejor.	Se usa una máscara de subred (ej. [redacted]) para separar la parte de red y la de host en una IP. Reduce tráfico broadcast y mejora seguridad.	Una empresa tiene [redacted] para oficinas y [redacted] para servidores — tráfico separado, misma red física.
DNS	La "guía telefónica" de internet: convierte nombres como *google.com* en direcciones IP.	Jerarquía de servidores (root → TLD → autoritativo) que resuelven queries mediante registros A, CNAME, MX, etc. Respuestas cacheadas por TTL.	Escribís [redacted] → tu PC pregunta al DNS → recibe [redacted] → recién entonces se conecta al servidor.

II)

El modelo OSI es un estándar conceptual que divide la comunicación en red en **7 capas independientes**. No es un protocolo real — es un *framework* para entender y diseñar cómo se comunican los sistemas.

Sirve para tres cosas concretas:

1. **Diagnosticar problemas** — si hay fallo de conexión, podés aislar en qué capa está el problema (¿cable? ¿IP? ¿app?).
2. **Diseñar redes** — cada capa tiene responsabilidad propia, entonces podés cambiar tecnología en una sin romper las demás.
3. **Hablar un lenguaje común** — fabricantes, devs y admins usan las mismas capas como referencia.

visualize show_widget

Una cosa clave del modelo es la **encapsulación**: cuando enviás datos, cada capa va agregando su propia cabecera (header) al paquete antes de pasarlo a la de abajo. Al llegar al destino, el proceso es inverso — cada capa lee y quita su cabecera hasta que la app recibe el dato original.

III)

DNS es básicamente la "guía de contactos" de internet. Las computadoras se comunican con direcciones IP (números), pero los humanos recordamos nombres. DNS traduce uno al otro.

¿Por qué no usamos IP directamente? Porque las IPs cambian, son difíciles de recordar, y un solo nombre puede apuntar a miles de servidores distintos según tu ubicación. `google.com` puede resolverse a IPs diferentes dependiendo de si estás en Chile, Europa o Asia.

Acá el flujo completo de lo que pasa cuando escribís `google.com`:

visualize show_widget

Dos datos que vale la pena destacar:

Todo ese proceso — pasos 3 al 6 — ocurre en **milisegundos**. Y la mayoría de las veces ni siquiera llega al paso 3 porque la respuesta ya está cacheada en tu máquina o en el resolver.

El `TTL` (Time To Live) es el parámetro que controla cuánto tiempo se guarda esa caché. Si Google quiere cambiar de servidor, baja el TTL primero para que el cambio se propague rápido.

IV)

🧩 4. Subnets: El mapa de tu red

Una **subred** (o subnet) es, en términos simples, una red dentro de una red. Es la segmentación lógica de una red física o virtual más grande para organizar y dirigir el tráfico de manera eficiente.

¿Qué es y para qué sirve?

- **Definición:** Es una división de una dirección IP (como una red Clase C) en trozos más pequeños.
- **Función:** Permite que los paquetes de datos viajen solo por el segmento de red donde se encuentra el destino, en lugar de "gritar" a todos los dispositivos de la red principal.

¿Por qué dividir redes? (Los 3 pilares)

1. **Seguridad (Aislamiento):** Puedes separar dispositivos sensibles (bases de datos) de dispositivos expuestos (servidores web). Si uno se ve comprometido, el atacante no tiene acceso automático a toda la red.
2. **Rendimiento:** Reduce el tráfico de "broadcast". Al haber menos dispositivos en una subred, hay menos "ruido" y colisiones de datos, lo que acelera la comunicación.
3. **Organización:** Permite agrupar dispositivos por función, departamento o nivel de confianza, facilitando la administración de IPs.

👉 Ejemplo Developer: Local vs. Cloud

Para un desarrollador, entender esto es la diferencia entre una app que funciona y una que es hackeada en 5 minutos.

Contexto	Ejemplo de Subred
Red Local (On-premise)	**Subred de Invitados:** El WiFi que le das a las visitas (solo internet). **Subred de Staff:** Donde están las impresoras y archivos internos.
Red en Cloud (VPC)	**Subred Pública:** Tiene salida a internet. Aquí pones tu Balanceador de Carga (ALB) o tu servidor Nginx. **Subred Privada:** No tiene acceso directo desde internet. Aquí vive tu Base de Datos (RDS) y tus microservicios de backend.

V) 5. Conexión directa con desarrollo (CRÍTICO)

🛠️ Diagnóstico de Conexión: ¿Qué está fallando?

Cuando una app falla al intentar comunicarse, el error suele estar en uno de estos tres puntos. Aquí te explico cómo diferenciarlos:

Mis respuestas honestas

1) el servidor dejo de funcionar, la red esta caída, para la DNS puedes ser que no se encuentra.
2) no se como responder el dominio
3) perdida de datos en la net, mas que nada en la red no esta funcionando bien

1. "No resuelve el dominio" (El traductor falló)

- **Qué pasa:** Tu app pregunta "¿Cuál es la IP de `mi-app.com`?" y el sistema responde "No tengo idea".
- **Causa:** El servidor **DNS** no responde, el dominio está mal escrito o el registro DNS no existe.
- **Relación: DNS**. Es como intentar buscar un nombre en una agenda telefónica y que la página esté arrancada. Sin la IP, la red no sabe a dónde ir.

2. "No encuentra el servidor" (La dirección no existe)

- **Qué pasa:** Ya tienes la IP (ej. `192.168.1.50`), pero al intentar llegar, no hay respuesta.
- **Causa:** Como bien dijiste, el **servidor dejó de funcionar** (está apagado) o la **red está caída**. También puede ser que la IP sea incorrecta.
- **Relación: IP**. Sabes a qué casa quieres ir, pero cuando llegas al terreno, la casa ya no está o el camino está cortado por un derrumbe.

3. "No puede conectarse" (La puerta está cerrada)

- **Qué pasa:** Encuentras el servidor, llegas a él, pero la conexión es rechazada.
- **Causa:** No es tanto pérdida de datos, sino un **bloqueo de seguridad**. Lo más común es un **Firewall** cerrando el puerto (ej. el puerto 80) o que el servicio dentro del servidor no está encendido.
- **Relación: Red / Firewall**. Llegaste a la casa correcta, pero la puerta tiene candado y no tienes la llave (o nadie atiende el timbre).

(un extra para mi)🛡️ VPN: El túnel invisible

Una **VPN** (Virtual Private Network) permite que tu dispositivo se conecte a una red privada a través de una red pública (Internet), como si estuvieras físicamente allí.

¿Cómo funciona técnicamente?

1. **Cifrado (El Escudo):** Tus datos se "meten" en un sobre cifrado. Si alguien intercepta el paquete en internet, no puede leer qué hay dentro.
2. **Encapsulamiento (El Túnel):** Tu paquete de datos original (con una IP privada de tu empresa, por ejemplo) se mete dentro de otro paquete con una IP pública. A esto se le llama *tunneling*.
3. **Punto de llegada:** El paquete viaja por internet hasta un **Servidor VPN**. Este servidor quita el "sobre" externo y deja salir el paquete original dentro de la red privada.

VI)

¿Qué significan estos errores?

Error	Traducción Técnica	¿Dónde está el problema?
"DNS not found"	No se pudo traducir el nombre (URL) a una dirección IP.	**Registros DNS / ISP:** El dominio no existe, expiró o el servidor DNS está fallando.
"Host unreachable"	Se conoce la IP, pero no hay una ruta física o lógica para llegar a ella.	**Networking / Enrutamiento:** El servidor está apagado, el cable está desconectado o el Router no sabe cómo llegar a esa subred.
"Network error"	Es un término "paraguas". Generalmente significa que la conexión se interrumpió a mitad del proceso.	**Firewalls / Timeouts:** Puede ser una pérdida de paquetes, un firewall bloqueando el tráfico o saturación de la red.

¿Es el DNS?

- Usa `nslookup` o `dig` seguido de tu dominio. Si no devuelve una IP, el problema es el nombre.

¿El servidor responde?

- Haz un `ping` a la IP. Si hay respuesta, el servidor "está vivo" y la red funciona. Si dice *Timeout*, el host es inalcanzable.

¿El puerto está abierto?

- Si el `ping` funciona pero la app no, usa `telnet` o `nc` (netcat) al puerto específico (ej. 80 o 443). Si falla aquí, es un **Firewall** o el servicio está apagado.

¿Es un problema de permisos (Nube)?

- Si usas AWS o Azure, revisa los **Security Groups** o las **ACLs** de la subred. Es muy común olvidar abrir el tráfico de entrada para el puerto de tu app.

1. ¿Es problema de Red? (Ojo con esto ⚠️)

- **Tu respuesta:** "No puede ser red, o no podría estar deployada".
- **La realidad: Sí puede ser la red.** * **Por qué:** El camino que usas para *deployar* (subir tu código vía GitHub Actions, SSH o FTP) es una **conexión de salida o administración**. El camino que usa el usuario es una **conexión de entrada**.
    - **El culpable:** El **Security Group** (en AWS) o el **Firewall**. Puedes tener permiso para "escribir" en el servidor, pero si el puerto 80/443 (HTTP/S) está cerrado para el mundo, nadie entrará.

2. ¿Es problema de DNS?

- **Tu respuesta:** "Puede ser, ya que nadie puede acceder".
- **La realidad: Es el primer sospechoso.**
    - **La prueba:** Si intentas entrar por la **IP directa** y la app carga, pero por el **nombre de dominio** no, entonces el DNS está mal configurado o aún se está propagando (los famosos registros A o CNAME).

3. ¿Es problema de Configuración?

- **Tu respuesta:** "Puede ser la configuración".
- **La realidad: ¡Bingo!** Aquí entran los errores "invisibles":
    - **Puerto equivocado:** Tu app corre internamente en el puerto `3000`, pero tu servidor web (Nginx/Apache) está buscando el tráfico en el `8080`.
    - **Host Binding:** Tu app está configurada para escuchar solo en `127.0.0.1` (localhost) en lugar de `0.0.0.0` (todas las interfaces), lo que significa que solo se habla a sí misma y no escucha a la red externa.

ANALOGÍA

🏢 El Modelo OSI (La Oficina de Correos)

Es como el proceso de enviar un paquete, donde cada departamento tiene una tarea única y obligatoria. Desde el empleado que redacta la carta hasta el camión que viaja por la carretera, nadie se salta su paso. Si el paquete no llega, las capas te permiten saber si falló el transporte o si el sobre estaba mal cerrado.

📖 DNS (La Lista de Contactos)

Es el traductor universal que convierte nombres humanos en coordenadas que las máquinas entienden. Tú no memorizas la latitud y longitud de tu pizzería favorita, simplemente buscas el nombre en tu GPS. Es el bibliotecario que toma el título del libro que pides y te dice exactamente en qué pasillo y estante está.

🏘️ Subnet (El Edificio de Departamentos)

Imagina un gran edificio con una sola dirección postal, pero dividido internamente por pisos y puertas. La subred es lo que evita que el repartidor de pizza toque en 100 casas antes de encontrar la tuya. Organiza el caos para que los vecinos del piso 1 no escuchen los gritos de los del piso 10, manteniendo el orden.

☁️ Conceptos de Infraestructura Cloud

1. ¿Qué es una VPC (Virtual Private Cloud)?

Es tu **parcela privada** dentro de la nube pública. Imagina que el proveedor de cloud es un continente enorme y la VPC es tu urbanización privada con su propia seguridad, calles y reglas.

- **Para qué sirve:** Para aislar tus recursos. Nadie puede entrar a tu VPC a menos que tú configures una puerta (Internet Gateway o VPN).
- **Relación:** Dentro de una VPC es donde creas las **Subredes** (públicas y privadas).

2. IP Pública vs. IP Privada

Es la diferencia entre tu dirección postal externa y el número de habitación de un hotel.

- **IP Pública:** Es la dirección visible en todo Internet. Es única en el mundo.
  - *Ejemplo:* La IP que apunta a tu dominio `www.tu-app.com`.
- **IP Privada:** Es la dirección interna dentro de tu red o VPC. Solo sirve para que tus servidores hablen entre ellos.
  - *Ejemplo:* Tu servidor de backend conectándose a tu base de datos.
  - **Clave:** Las IPs privadas no cuestan dinero y son más seguras porque no se puede llegar a ellas desde fuera.

3. ¿Qué es un Load Balancer (Balanceador de Carga)?

Es el **"Director de Tráfico"** de tu aplicación. Si tienes mucha gente entrando a tu app, un solo servidor podría explotar.

- **Cómo funciona:** El Load Balancer recibe todas las peticiones (en la IP pública) y las reparte de forma equitativa entre varios servidores que están por detrás (usando sus IPs privadas).
- **Ventaja:** Si un servidor se rompe, el balanceador simplemente deja de enviarle tráfico y redirige todo a los servidores que sí funcionan. Tu app nunca se cae.

💡 Resumen para tu "Mental Map"

> "Un usuario busca tu **DNS**, el cual apunta a la **IP Pública** de tu **Load Balancer**. Este reparte el tráfico dentro de tu **VPC** hacia diferentes **Subredes** donde viven tus servidores con **IPs Privadas**."